Cloud computing

Course notes

Daniele Besozzi

A.A. 26/27

0 Contents

1 Introduction and definition 3
1.1 Essential characteristics 3
1.2 Why cloud computing? 4
1.3 Data centers and cloud computing 4
1.4 Virtual infrastructure manager (VIM) 5
2 Service models (SPI) 6
2.1 Infrastructure as a Service (IaaS) 6
2.1.1 Resource Management Interface 7
2.1.2 System Monitoring Interface 7
2.1.3 Properties of IaaS 7
2.2 Platform as a Service (PaaS) 8
2.2.1 Properties of PaaS 9
2.3 Software as a Service (SaaS) 10
2.3.1 Properties of SaaS 11
2.4 The SPI model 11
3 Deployment models 13
3.1 Public Cloud 13
3.2 Private Cloud 13
3.3 Community Cloud 13
3.4 Hybrid Cloud 13
4 Characteristics 14
4.1 Utility Computing 14
4.1.1 Service oriented architecture (SOA) 15
4.2 Scalability and Elasticity 15
4.2.1 Traditional provisioning 16
4.2.2 Dynamic provisioning 16
4.3 Availability and Reliability 17
4.3.1 Fault tolerance 17
4.3.2 System resilience 18
4.3.3 System Security 18
4.4 Manageability & Interoperability 18
4.4.1 System Monitoring 18
4.4.2 Billing System 19
4.5 Performance optimization 19
4.5.1 Economical aspects of cloud computing 19
5 Virtualization 20
5.1 Computer architecture 20
5.1.1 Emulation 20
5.2 Virtualization and Virtual Machines 20
5.2.1 Protection rings 21
5.3 Virtualization of resources 22
5.3.1 Memory virtualization 22
5.3.2 I/O virtualization 23
5.3.3 Network virtualization 23
5.4 Advantages of virtualization 23

5.4.1 Workload isolation 23
5.4.2 Workload migration 24
5.4.3 Consolidation 24
5.5 Taxonomy of hypervisors 24
5.6 Virtualization techniques 24
5.6.1 Full Virtualization using binary translation 24
5.6.2 Para Virtualization (a.k.a. OS assisted Virtualization) 25
5.6.3 OS-Level virtualization (Containerization) 25
6 Amazon infrastructure 26
6.1 Key concepts 26
6.1.1 S3 26
6.1.2 EBS 26
6.1.3 Security group 27
6.2 EC2 27
6.2.1 Ephemeral resources 27
6.2.2 Persistent Resources 28
6.2.3 Pricing model 28
6.2.3.1 Purchase options 28

1 Introduction and definition

We can define cloud computing in many ways, but the most important thing is to understand the basic definition. A Cloud is a type of distributed system consisting of:

- **interconnected** and **virtualized** computers
- **provisioned on demand** and
- **accessible** through a network.

Alternatively, we can say that cloud computing is:

- a **user experience**, in a sense that it is a way to provide services to users over the network, and
- an **infrastructure management methodology**, because it is a way to manage large numbers of highly virtualized resources so that they can be automatically aggregated to deliver services.

We can give a more formal definition by the NIST (National Institute of Standards and Technology):

> **Definition 1.1** *(Cloud computing).* Cloud computing is a **model** for enabling **convenient**, **on-demand network access** to a **shared** pool of **configurable computing resources** (e.g., networks, servers, storage, applications, and services) that can be **rapidly provisioned and released with minimal management** effort or service provider interaction.

This cloud model promotes availability and is composed of:

- five essential characteristics,
- three service models, and
- four deployment models.

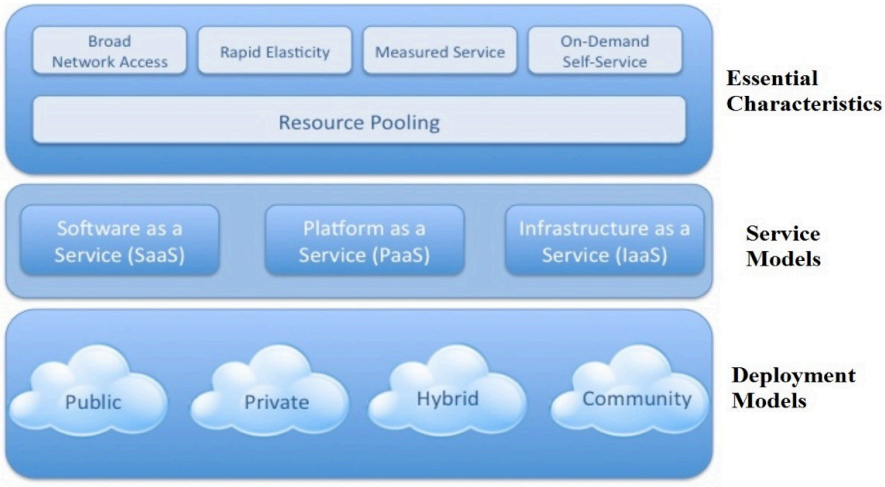


Figure 1: Cloud computing characteristics and models

This gives us a technological, economical, and societal model.

1.1 Essential characteristics

The essential characteristics of cloud computing are:

- **On-demand self-service**:
  - A consumer can unilaterally provision computing capabilities such as server time and network storage as needed automatically, without (or with little) requiring human interaction with a cloud provider.

- **Broad network access**:
  - Capabilities are available over the network and accessed through standard mechanisms that promote use by heterogeneous thin or thick client platforms (e.g., mobile phones, laptops, and PDAs) as well as other traditional or cloud- based software services.
- **Resource pooling**:
  - The provider's computing resources are pooled to serve multiple consumers using a multi-tenant model, with different physical and virtual resources dynamically assigned and reassigned according to consumer demand.
- **Rapid elasticity**:
  - Capabilities can be rapidly and elastically provisioned to quickly scale up; and rapidly released to quickly scale down, automatically.
  - To consumers, the capabilities available for provisioning often appear to be unlimited and can be purchased in any quantity at any time.
- **Measurable service**:
  - Cloud systems automatically control and optimize resource usage by leveraging a metering capability at some level of abstraction appropriate to the type of service.
  - Resource usage can be monitored, controlled, and reported - providing transparency for both the provider and consumer of the service.

1.2 Why cloud computing?

The cloud shape is often used in network diagrams to represent a "black box" network. The access **interfaces are known, but not the internal structure**. A cloud is opaque and considered very large and distant. As in the sky there are **different types of clouds** so there are **different types of services** and **features** offered in the cloud. The internal implementation of these services is **opaque**.

Clouds may overlap, that is, different models of clouds can be implemented by the same vendor. Clouds can **dynamically change shape**. Applications and services in cloud can dynamically **adapt**.

1.3 Data centers and cloud computing

Data centers are buildings that house hundreds of thousands of servers that simultaneously support many services and applications. Data centers are built on a large scale to reduce maintenance costs, but not very large (also to limit blast radius).

Data centers exploit **virtualization** massively to run applications.

A data center requires:

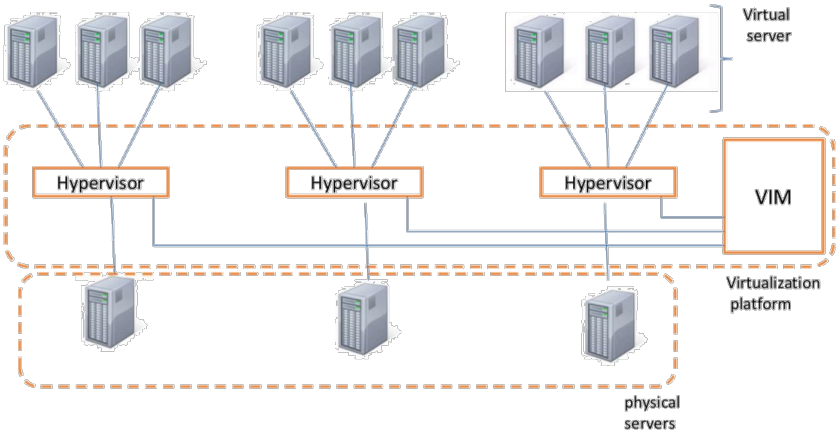
- **Power supply** (generator system to overcome possible power failures).
- Continuous **cooling** system.
- **Security system** (fire alarms, adequate ventilation to eliminate smoke, authorized access, video cameras, security code access, biometric access, card reader access, locked cabinets…).
- **High-speed network**, within the data center, between data centers and to the Internet Often optical fiber, dedicated and proprietary lines, including intercontinental lines.

> **Definition 1.2** *(Availability zone).* Every Availability Zone is an independent data center with its own power grid and network so that it is protected from possible failures in other data centers

Definition 1.3 *(Region).* A Region is a set of Availability Zones that are geographically close to each other and connected by low-latency links. A Region is isolated from other Regions, so that a failure in one Region does not affect other Regions

1.4 Virtual infrastructure manager (VIM)

A key challenge cloud providers face in building an infrastructure is managing physical and virtual resources, i.e., servers, storage and networks, holistically. **Resource orchestration** must be performed in a way that quickly and dynamically delivers resources to users and applications. The software tool responsible for the orchestration is called a **Virtual Infrastructure Manager (VIM)**. This type of software resembles a traditional operating system ("cloud operating system") - but instead of dealing with a single computer, it aggregates resources from multiple computers, presenting a uniform view to the user and applications.



2 Service models (SPI)

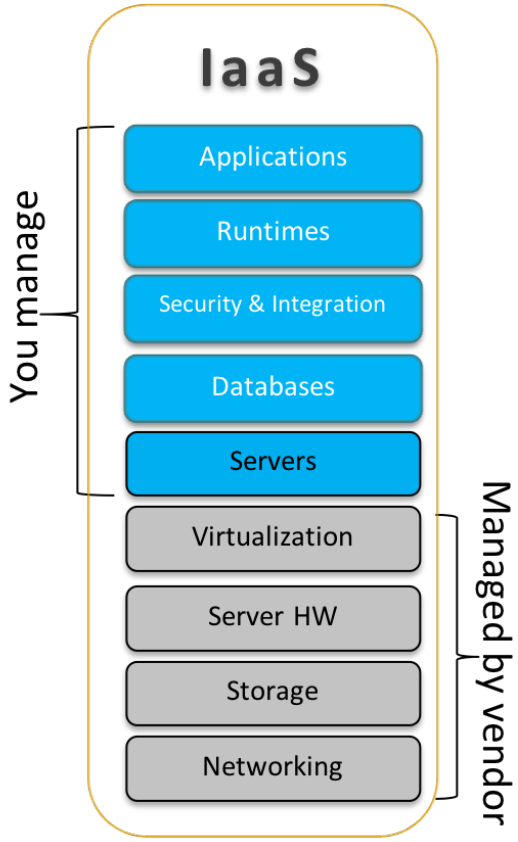
A service model is, in this scenario, a high-level characterization (model) of the cloud that **focuses primarily on the services offered and the type of users**. The set of service models defines a possible **taxonomy**.

2.1 Infrastructure as a Service (IaaS)

The capability provided to the consumer (user or application) is to provision:

- virtual machines,
- storage,
- networks, and
- other fundamental computing resources

The consumer can deploy and run arbitrary software on the virtualized resources, which can include OS and applications. The consumer **does not manage or control the underlying cloud infrastructure** but has **control over operating systems, storage, deployed applications**, and possibly limited control of selected networking components.



This kind of technique is enabled by **virtualization**.

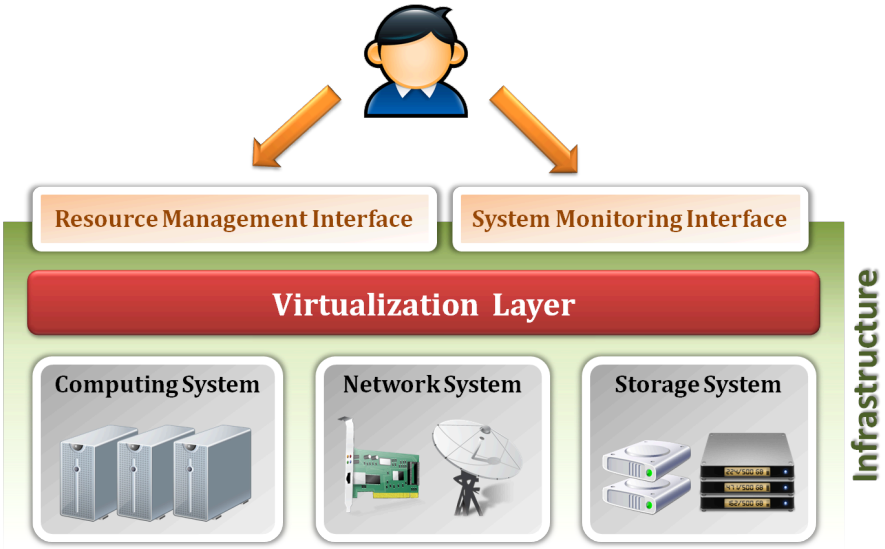


Figure 4: IAAS architecture

2.1.1 Resource Management Interface

RMI Enables the customer to obtain and manage several types of virtualized resource:

- **Virtual Machine**: IaaS provides the basic virtual machine operations, such as creation,, suspension, resumption and termination.
- **Virtual Storage**: IaaS provides the basic virtual storage operations, such as space allocation, space release, data writing and data reading.
- **Virtual Network**: IaaS provides the basic virtual network operations, such as IP address allocation, domain name register, connection establishment and bandwidth provision.

2.1.2 System Monitoring Interface

There are several metrics to be monitored:

- **Virtual Machine**: As an IaaS provider, we should be able to monitor some system states of each virtual machine, such as CPU loading,, memory utilization, IO loading and internal network loading
- **Virtual Storage**: As an IaaS provider, we should be able to monitor some storage states of each virtual storage, such as virtual space utilization, data duplication and storage device access bandwidth, ...etc.
- **Virtual Network**: As an IaaS provider, we should be able to monitor some network states of each virtual network, such as virtual network bandwidth, network connectivity and network load balancing

2.1.3 Properties of IaaS

- Manageability and Interoperability
  - System control, billing, API
- Availability and Reliability
  - Multiple copies of VMs, storage redundancy
- Scalability and Elasticity
  - On demand resource (ex. VMs) provisioning

> This technique is enabled by **virtualization**



Figure 5: Properties of IaaS

2.2 Platform as a Service (PaaS)

The capability provided to the consumer is to **develop and deploy** onto the cloud applications created using **programming languages and tools supported by the provider**. The **consumer** does not manage or control the underlying cloud infrastructure including network, servers, operating systems, or storage, but has **control over the deployed applications and possibly application hosting environment configurations**.

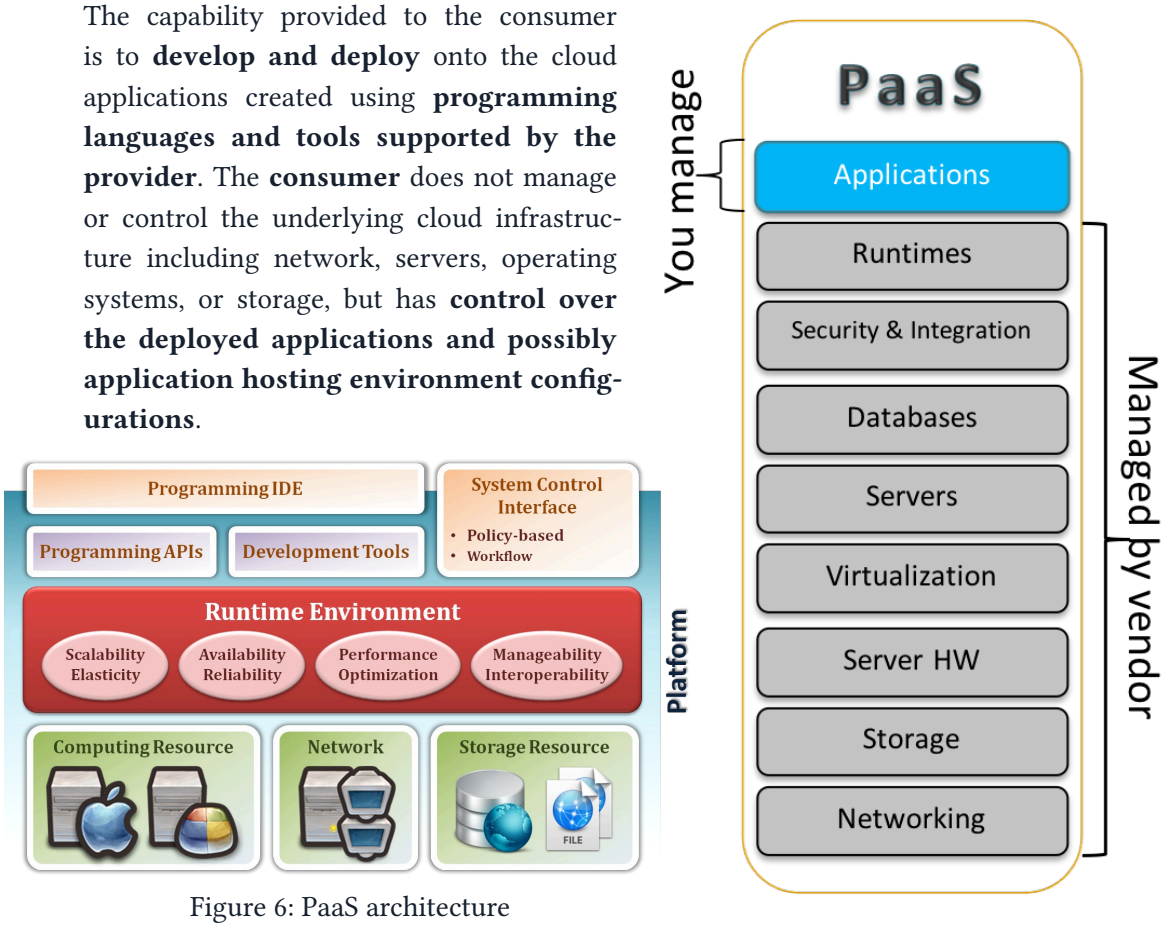


Figure 6: PaaS architecture

> This technique is enabled by **runtime environment**, that is acollection of software services available. Usually implemented by a collection of program libraries and an application execution engine

PAAS offers different services:

- Users make use of **programming IDE** to **develop their service** over PaaS.

  - This IDE should integrate the full functionalities supported from the underling runtime environment. This IDE should also provide development tools, such as profiler, debugger and testing environment.
  - The programming APIs supported from runtime environment may be heterogeneous across different cloud providers, but there are still some common operating functions.
- **System Control Interface**
  - **Policy-Based Control**
    - Typically described as a principle or rule to guide decisions and achieve rational outcome(s)
    - Make the decision according to some requirements
    - If <condition> then <action>
  - **Workflow Control**
    - Describes the flow of installation and configuration of resources
    - Delivers speedy and efficient construction and management of cloud resources

2.2.1 Properties of PaaS

Common properties in Runtime Environment :

- Manageability and Interoperability
- Performance and Optimization
- Availability and Reliability
- Scalability and Elasticity



Figure 8: Properties of PaaS

2.3 Software as a Service (SaaS)

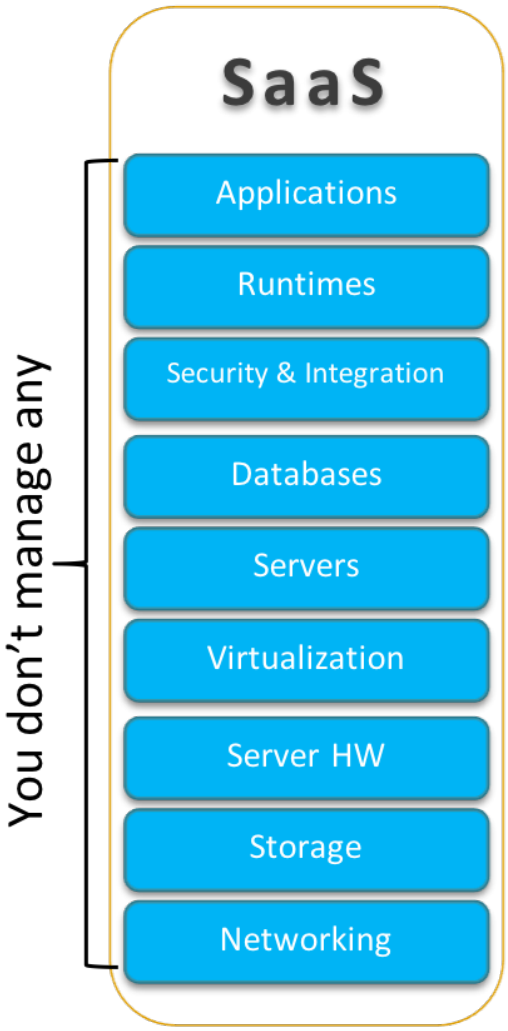
The capability provided to the consumer is to use the **provider's applications running on a cloud infrastructure**. The applications are accessible from various client devices through a thin client interface such as a web browser (e.g., web- based email).

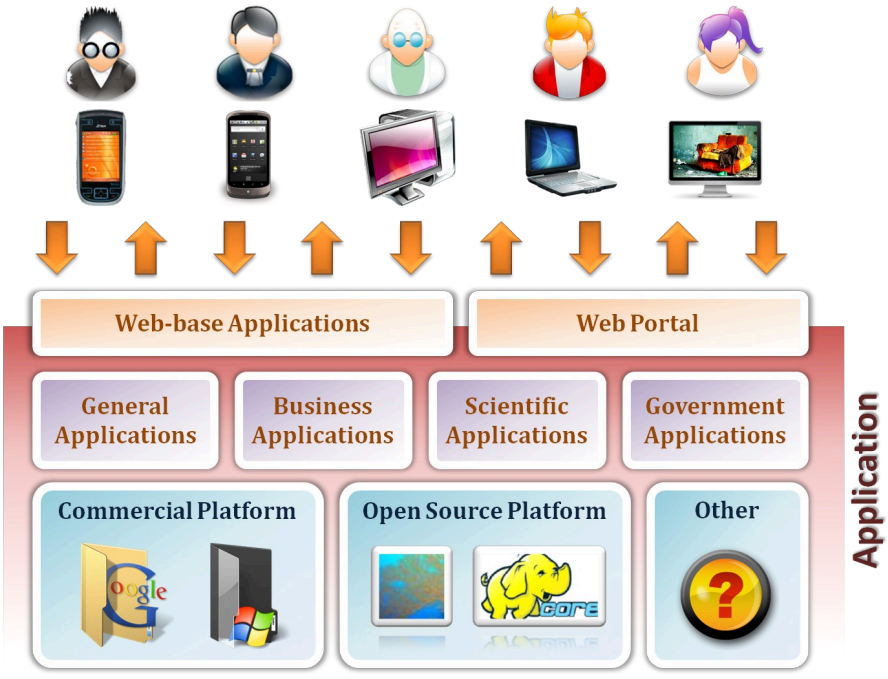
The **consumer does not manage or control the underlying cloud infrastructure** including network, servers, operating systems, storage, or individual application capabilities, except for limited user-specific application configuration settings.

> This technique is enabled by **web services**

Web 2.0 is the trend of using the full potential of the web:

- Viewing the Internet as a computing platform
- Running interactive applications through a web browser
- Leveraging interconnectivity and mobility of devices
- Enhanced effectiveness with greater human participation





Services can be provided in many forms:

- **web-base applications**. Conventional applications should translate their access interface onto web-based platform. Applications can have different domains:
  - General Applications : Applications which are designed for general propose, such as office suit, multimedia and

  instant message.
  - Business Applications : Application which are designed for business propose, such as ERP, CRM and market

  trading system.
  - Scientific Applications : Application which are designed for scientific propose, such as aerospace simulation and

  biochemistry simulation.
  - Government Applications : Applications which are designed for government propose, such as national medical

  system and public transportation system service.
- **Web portal**: Apart from the standard search engine feature, web portals offer other services such as e-mail, news, stock prices, information, databases and entertainment. Portals provide a way for enterprises to provide a consistent look and feel with access control and procedures for multiple applications and databases, which otherwise would have been different entities altogether.

2.3.1 Properties of SaaS

The properties offered by SaaS are the ones offered by the internet, that is **accessibility** and **portability**.

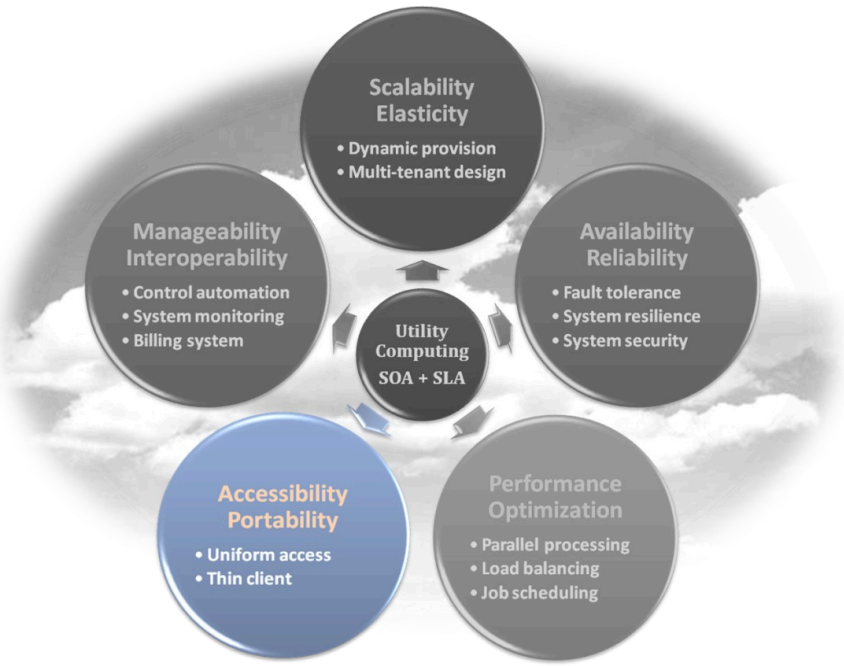


Figure 11: Properties of SaaS

2.4 The SPI model

The SPI model (SaaS, PaaS, IaaS) is the most common classification of the cloud services. But, be aware that the **real cloud is blurry**. The provider can offer different services at different levels, and it is not always easy to classify them.

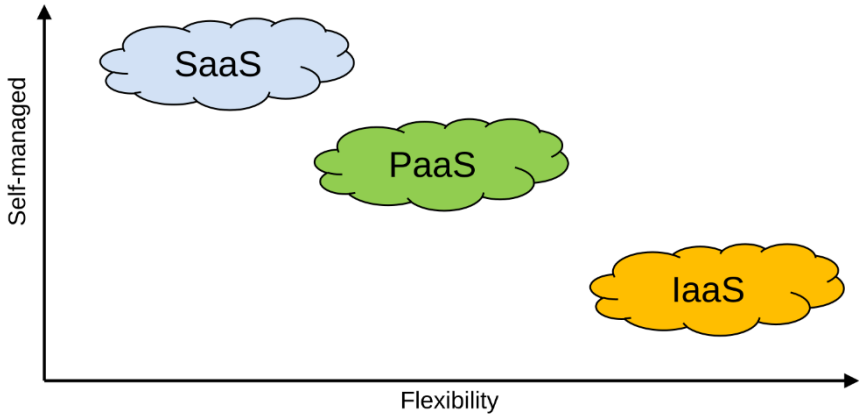


Figure 12: The SPI model

3 Deployment models

A cloud deployment model is defined according to **where the infrastructure resides** and **who has control over that infrastructure**. Deciding which deployment model you will go with is one of the most important cloud deployment decisions you will make.

There are four primary cloud models:

- Public Cloud
- Private Cloud
- Community Cloud
- Hybrid Cloud

Differences lie primarily in the **access**, **size** and **ownership** of published cloud services

3.1 Public Cloud

The cloud infrastructure is made **available to the general public** and is owned by an **organization selling cloud services**. Also known as external cloud or multi-tenant cloud, this model essentially represents a cloud environment that is openly accessible. Basic characteristics:

- **Homogeneous** infrastructure
- Common policies
- Shared resources and multi-tenancy
- Leased or rented infrastructure
- Economies of scale

3.2 Private Cloud

The cloud infrastructure is **operated solely for one organization**. It may be managed by the same organization or a third party and may exist **on premise or off premise**.

Also referred to as internal cloud or on-premise cloud, a private cloud intentionally limits access to its resources to consumers that belong to the organization that owns the cloud. Basic characteristics :

- **Heterogeneous** infrastructure
- Customized and tailored policies
- Dedicated resources
- In-house infrastructure
- End-to-end control

3.3 Community Cloud

The cloud infrastructure is **shared by several organizations** It supports a specific community that has shared concerns (e.g., mission, security requirements, policy, and compliance considerations). Access to a community cloud environment is **typically restricted to the members of the community**.

3.4 Hybrid Cloud

The cloud infrastructure is a **composition** of two or more clouds (private, community, or public) that remain unique entities but are **bound together by standardized or proprietary technology** that enables data and application portability (e.g., cloud bursting for load-balancing between clouds). Typical example: Private-Public for scalability.

4 Characteristics

Cloud computing is a paradigm of computing, a new way of thinking about IT industry but **not any specific technology**.

Central ideas:
- Utility Computing
- SOA - Service Oriented Architecture
- SLA - Service Level Agreement

Enabling techniques:
- Virtualization
- Parallelized and distributed computing
- Service-oriented computing
- Autonomic Computing

- **scalability → elasticity**
  - ability → dynamic resource mechanism
- **availability → reliability**
  - service property → fault-tolerance mechanism
- **manageability → interoperability**
  - ability to manage → ability to integrate heterogeneous systems
- **accessibility → portability**
  - ability to access/use → ability to move across environments
- **performance → optimization**
  - desired outcome → improvement mechanism

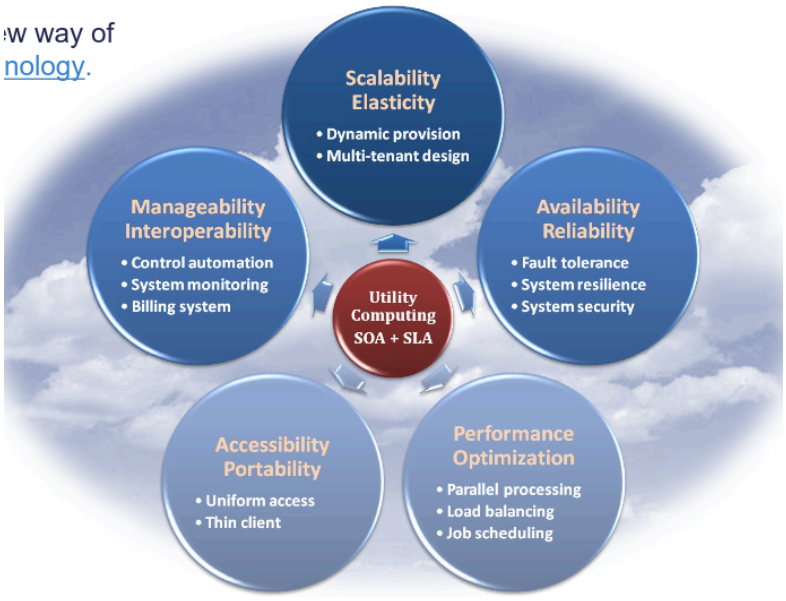


Figure 13: Cloud computing characteristics

4.1 Utility Computing

Definition 4.1 *(Utility Computing).* Utility Computing is a computing model in which computing resources, such as processing power, storage and networking are provided to users as metered services, similar to traditional public utilities like electricity, water, or gas.

Users do not own or manage the underlying hardware or infrastructure. Instead, they access computational resources on demand and pay only for the amount they actually consume.

Users do not care about **how** the work is done. Instead, they only concern about what they can get.

Users do not care about **what** the provider does. Instead, they only concern about the quality of service.

Users do not **own** the physical infrastructure. Instead, they only pay per usage

Users typically only care about the **quality of service**.

Utility computing is a model for **service provisioning**: the service provider makes **computing resources** and **infrastructure management available** to the customer as needed, and **charges them for specific usage** rather than a flat rate.

Like other types of on-demand utility, the utility model seeks to **maximize the efficient use of resources** (provider side) and/or **minimize associated costs** (provider and user).

4.1.1 Service oriented architecture (SOA)

> **Definition 4.2** *(Service Oriented Architecture).* Service Oriented Architecture (SOA) is essentially a collection of services which communicate with each other.

This type of architecture provides a loosely-coupled suite of services that can be used within multiple business domains. Usually it is implemented using the Web Service model.

> **Definition 4.3** *(Quality Of Service).* **QoS** (in the cloud context) denotes the **levels of performance**, **reliability**, and **availability** offered by the platform or infrastructure that hosts a reference application.

> **Definition 4.4** *(Service Level Agreement).* **A service-level agreement (SLA)** is a **contract between a service provider and a customer** that specifies, usually in measurable terms (QoS), what level of service the provider will furnish. Defines the **non-functional characteristics** guaranteed by the service. An SLA includes several **SLOs (Service Level Objectives)** that define the quality of service to be guaranteed through specific metrics.

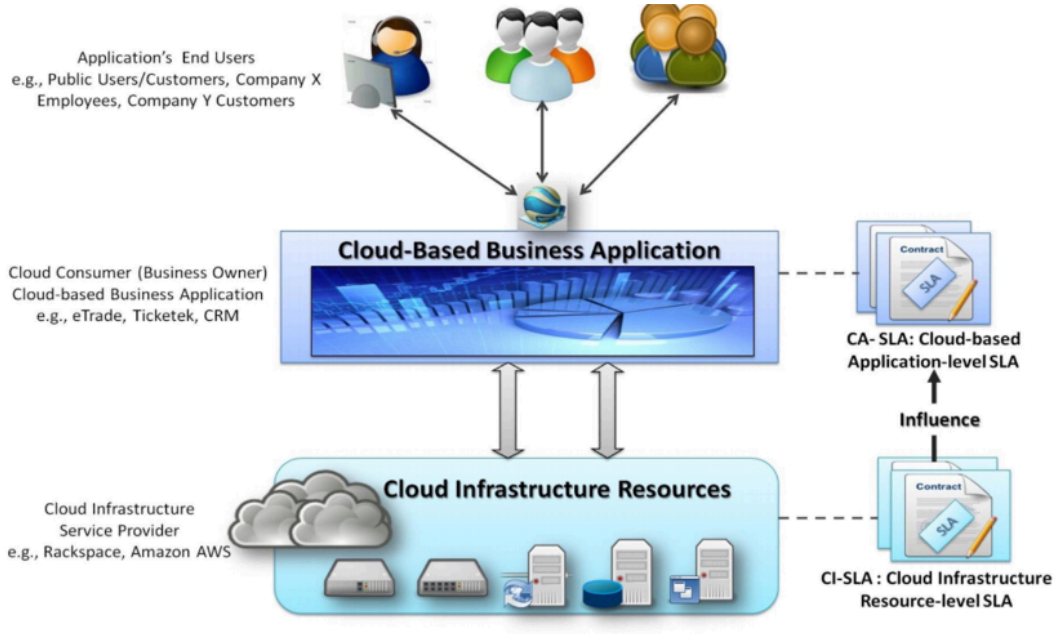


Figure 14: Service Level Agreement (SLA) and cloud infrastructure.

4.2 Scalability and Elasticity

- **Scalability** is a property of a system, a network, or a process, indicates its ability to either handle growing amounts of work in a graceful manner or to be readily enlarged (empowered).

- **Elasticity** is the ability to enforce adaptive "autonomic" actions to enable scalability.

These properties are achieved by **scaling**:

vertical (up or down):
- More powerful computing resources or bigger servers
- Single point of failure
- Human intervention
- Reasonable management overhead

horizontal (out or in):
- More computing resources (e.g., servers)
- Reliable – fail-over scenario (there is at least one node to take over)
- It can be fully automated (especially in the cloud)
- Growing management complexity

Definition 4.5 *(Dynamic provisioning).* Dynamic Provisioning describes a complex networked computing environment where **computing instances** are **provisioned** or **deployed** from an administrative console or client application.

The system might enable rules of automatizing the provisioning. In traditional computing model, two common problems:
- **Loss revenue / Loss Users** because of under provision (underestimation of system utilization).
- Organizations **overestimate** system utilization to ensure they can handle peak loads.

4.2.1 Traditional provisioning

Traditional provisioning involves allocating resources based on peak capacity requirements. Organizations **overestimate** system utilization to ensure they can handle peak loads. This conservative approach results in:
- Low average utilization: Resources sit idle during non-peak periods
- Higher operational costs: Paying for capacity that isn't being used most of the time
- Capital inefficiency: Significant investment in infrastructure that delivers poor return on investment.

This traditional model contrasts with modern cloud computing approaches, where resources can scale dynamically to match demand, reducing waste and improving cost efficiency.

4.2.2 Dynamic provisioning

Cloud resources can be be provisioned dynamically in response to actual demand patterns (Elasticity):
- **Meet seasonal demand variations**: Handle predictable cycles like holiday shopping periods or tax season.
- **Meet demand variations between different industries**: Adapt to sector-specific patterns and business cycles.
- **Meet burst demand for some extraordinary events**: Scale rapidly for unexpected spikes like viral content or flash sales.

With dynamic provisioning:
- Capacity (red dashed line) now closely follows the demand curve
- Resources scale up and down automatically to match actual needs
- The gray shaded area (waste) is dramatically reduced
- Capacity and demand are nearly aligned throughout the entire cycle

Organizations that do provisioning **without using cloud technologies** must commit to fixed capacity investments based on imperfect forecasts, inevitably leading to periods of both over-provisioning (waste) and under-provisioning (service issues). This inflexibility makes it nearly impossible to optimize costs while maintaining adequate performance.

> **Cloud provisioning** creates near-perfect alignment between capacity and demand throughout the entire lifecycle, eliminating both waste and under-provisioning while converting fixed capital costs into variable operational expenses. This methodology gives us:
> - No under-supply
> - Reduction of over-supply
> - Reduction of initial investments
> - Improved Performances

4.3 Availability and Reliability

- **availability**:
  - is the proportion of time a system is in a functioning condition expressed as 100% minus unavailability
  - Cloud infrastructure usually provides high availability Ex. “Five Nines” system would statistically provide 99.999% availability
- **reliability**:
  - the ability of a system to perform its required functions under stated conditions for a specified period of time.

How to achieve these properties (at application level)?

- Fault tolerance system
- System resilience
- Reliable system security

4.3.1 Fault tolerance

Fault-tolerance is the property that enables a system to **continue operating properly** in the event of the **failure** of some of its components. If the operating quality decreases at all, the decrease must be proportional to the severity of the failure, as compared to a naively-designed system in which even a small failure can cause total breakdown.

Four basic actions:

- No **single point of failure**
  - One of the possible flaws in designing an application system
  - A part of a system which, if it fails, will stop the entire system from working
  - Preventive identification of the critical components of a system that would provoke a total systems failure in case of malfunction.
  - Can be prevented by **replication** of the critical components
- **Fault detection** of the failing component
  - **Fault Detection and Isolation (FDI)**: A subfield of control engineering which concerns itself with monitoring a system, identifying when a fault has occurred and pinpoint the type of fault and its location.
- **Fault containment** to prevent propagation of the failure
  - When a failure occurs, the system must be able to isolate the failure to the offending component.
  - To avoid other components to be taken down by the erratic or unexpected behavior
  - In the cloud usually faulty components are identified, isolated and terminated automatically
- Availability of **reversion** modes

Actions are implemented at Cloud level but the Cloud can help the designer to implement them at application level.

4.3.2 System resilience

Resilience is the ability to provide and maintain an **acceptable level of service** in the face of faults and challenges to normal operation. Resiliency pertains to the system's ability to **return to its original state** after encountering troubles. If a failure event knocks a system offline, a highly resilient system will return back to work and function as planned as soon as possible.

> **Definition 4.6** *(Disaster Recovery).* Disaster recovery is the set of **policies** and **procedures** related to be **prepared** for and **execute** the recovery after a disaster.

Some common strategies involve the use of backups of the whole system to an off-site location done periodically and the preparation of local mirrored systems, surge protection and UPS (Uninterruptible Power Supply) systems to ensure that the system can continue to operate in the event of a power outage.

4.3.3 System Security

Cloud security is a sub-domain of computer security, network security, and, more broadly, information security. It refers to a broad set of **policies**, **technologies**, and controls deployed to **protect data**, **applications**, and the associated **infrastructure** of cloud computing.

Important security and privacy issues :

- **Data Protection**
  - To be considered protected, data from one customer must be properly segregated from that of another.
- **Identity Management**
  - Every enterprise will have its own identity management system to control access to information and computing resources.
- **Application Security**
  - Cloud providers should ensure that applications available as a service via the cloud are secure.
- **Privacy**
  - Providers ensure that all critical data are masked and
  - that only authorized users have access to data in its entirety.

4.4 Manageability & Interoperability

Manageability:

- The ability of a system to be controlled either by means of self-control or by providing certain techniques to ease external controls
- Enterprise-wide administration of cloud computing systems.

Interoperability:

Interoperability is a property of a product or system to work with other products or systems, present or future, without any restricted access or implementation.

- Open interfaces / open access
- Web based interfaces
- Virtualization

These properties are achievable by system state monitoring and control automation.

4.4.1 System Monitoring

A System Monitor is a process within a distributed system for collecting and storing state data.

- Physical and virtual hardware state
- Resource performance metrics
- Network access patterns
- System logs

- ... etc

4.4.2 Billing System

Users pay per the **resources they used**. Cloud providers must

- determine the list of **service usage price**
- **record** the resource or service usage of each user, and then charge users by these records.
  - Get those information by means of monitoring system.
  - Automatically calculate the total amount of money which user should pay.
  - Automatically request money from use's banking account.

4.5 Performance optimization

This can be achieved by means of **load balancing**.

> **Definition 4.7** *(Load balancing).* Load balancing is a technique to **distribute workload evenly** across two or more computers, network links, CPUs, hard drives, or other resources, in order to get **optimal resource utilization**, maximize throughput, minimize response time, and **avoid overload**.

4.5.1 Economical aspects of cloud computing

Some aspects of cloud computing to be considered in an economic assessment:

- Model pay-as-you-go: no upfront costs for the cloud user - pay only for what you consume:
  - Form Capital Expenditure (**CAPEX**)
  - To Operational Expenditure (**OPEX**)

Elasticity gives risk transfer:

- No risks associated with incorrect sizing of the necessary infrastructure
- Provider absorbs infrastructure risk

Cloud give both the provider and the user the benefit of **economies of scale**:

- **Infrastructure Efficiency**: Large data centers achieve lower per-unit costs for compute, storage, networking,

electricity, cooling, and physical space

- **Cost Advantages**: providers pass savings to users through competitive pricing that small/medium organizations cannot match independently
- **Expertise Pooling**: Access to world-class skills, best practices, and cutting-edge technologies without building internal capabilities

5 Virtualization

First, we review the architecture of a computer, which is the basis of virtualization technology.

5.1 Computer architecture

Since the 1940's, computers have 5 classic components

- **Input** devices: Keyboard, mouse, cartridge, DVD ...
- **Output** devices: Display, printer ...
- **Storage** devices
  - Volatile **memory**: RAM (Random Access Memory)
  - Permanent **storage**: Magnetic, Optical, and Flash disks, ...
- Datapath, Control unit } Together, they are called the **Processor**

In the new concept of a computer, the **network** also is a new component.

The Central Processing Unit (**CPU**) loads programs from the Storage to the RAM and executes in **linear fashion** a succession of instructions (now you can call the programs processes). The "words" that the CPU understands belong to an **Instruction Set**.

An **Operating System** is a special program that controls the hardware and manages the **life-cycle of processes**. Several processes can run concurrently on the same HW, which is shared thanks to an OS that implements CPU time-sharing (**Multitasking**).

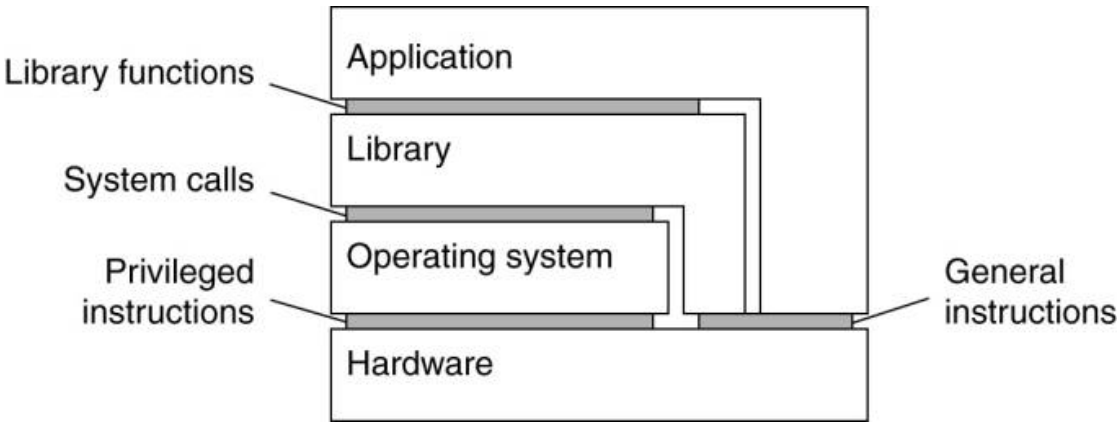


Figure 15: Layers of instructions and interaction with the hardware

We divide instructions by the level of abstraction:

- **Unprivileged machine instructions** (General instructions): an interface to the bare hardware that is available to any program
- **Privileged instructions**: provide an interface to the hardware for the OS
- **System calls**: provide an interface to the operating system for applications
- **APIs**: high-level programming endpoints that may exploits OS system calls

5.1.1 Emulation

Emulation is a technique where one computer system (the **host**) **mimics the hardware and instruction set** of a different computer system (the **guest**). It **translates** instructions from the guest architecture into instructions the host can execute, **recreating the complete behavior of the original system in software**.

5.2 Virtualization and Virtual Machines

Virtualization is a technology that creates **abstracted, logical instances of physical computing resources**. It uses a **hypervisor** (or Virtual Machine Monitor) to partition physical hardware and create multiple isolated virtual environments called **Virtual Machines** (VMs).

Each VM runs its own operating system and applications, believing it has dedicated hardware, while sharing the underlying physical resources, i.e., the ISA (Instruction Set Architecture).

Aspect	**Virtualization**	**Emulation**
Architecture	Same ISA (e.g., x86 on x86)	Different ISA (e.g., ARM on x86)
Performance	Near-native speed	Significantly slower
Translation	Minimal/no instruction translation	Full instruction translation required
Use case	Server consolidation, cloud computing	Legacy system preservation, cross platform testing
Example	VMware ESXi, KVM, Hyper-V	QEMU (in full emulation mode), DOSBox, game console emulators

Definition 5.1 *(Virtual machine).* A **virtual machine** provides a complete **virtualized hardware platform** to support running a complete operating system. The resources of the underlying physical machine can be shared among several virtual machines, each with its own operating system.

Virtual machines are created and managed by a **Hypervisor** (a.k.a. Virtual Machine Monitor) Examples: VirtualBox, KVM, VMware and Xen.

We can further distinguish between two types of virtual machines:

- **System virtual machines**: Provide a complete system platform that supports the execution of a full operating system (e.g., VMware, VirtualBox, KVM).
- **Process virtual machines** (also application virtual machines or managed runtime environments): Designed to run a single program, providing a platform-independent programming environment (e.g., Java Virtual Machine, .NET Common Language Runtime). They only virtualize the resources needed to ecevute a specific application.
  - **Process-level abstraction**: Virtualizes at the application layer, not the hardware layer
  - **Single application**: Each VM instance runs one application or process
  - **Platform independence**: "Write once, run anywhere" - code runs on any platform with the VM
  - **Managed execution**: Provides runtime services like memory management, garbage collection, and security
  - **Intermediate representation**: Applications are typically compiled to platform-independent bytecode, which the VM interprets or JIT-compiles

From here on, we will focus on system virtual machines, which are the most relevant for cloud computing.

Definition 5.2 *(Hypervisor).* An **Hypervisor** (or Virtual Machine Monitor) is a software that creates and manages virtual machines by abstracting and partitioning physical hardware resources. It leverages hardware virtualization features (such as Intel VT-x or AMD-V) provided by modern CPUs to efficiently manage virtual machines. Shares the available resources in such a way that each VM sees them as **dedicated**.

5.2.1 Protection rings

Modern CPUs define protection levels (rings) from Ring 0 (highest privilege) to Ring 3 (lowest privilege).

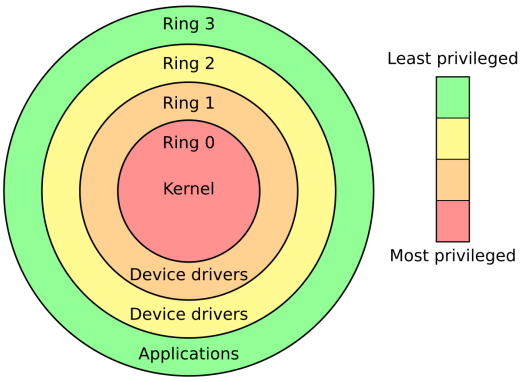


Figure 16: Protection rings

Modern CPUs define protection levels (rings) from Ring 0 (highest privilege) to Ring 3 (lowest privilege). Operating Systems kernels expect to run at Ring 0 to execute privileged instructions directly on hardware. Hypervisors need to intercept these instructions: early solutions used:

- trap-and-emulate.

> In a virtualized environment, the guest OS believes it has full control over the hardware, but it actually runs in a deprivileged mode (typically user mode or a less privileged ring). When the guest OS attempts to execute a privileged instruction, the CPU generates a trap (exception), transferring control to the hypervisor, which then emulates the instruction's behavior in a safe, virtualized manner.

- binary translation.

> Binary translation is a virtualization technique that dynamically scans, analyzes, and rewrites guest code at runtime to replace problematic instructions with safe equivalents that can be executed in a virtualized environment. It was the solution VMware developed to virtualize x86 architecture before hardware virtualization support existed.

Hardware-assisted virtualization (Intel VT-x, AMD-V) introduced new CPU modes that allow guest OSes to run as if they were in Ring 0, while still being controlled by the hypervisor. CPUs add a new mode that lets the **hypervisor run in "root mode"** while the **guest OS runs in "non-root mode"**. When the guest tries to execute a privileged instruction, the CPU automatically traps it and hands control to the hypervisor (Privileged instruction handling), but also:

- Efficient context switching between guest and host environments.
- Memory virtualization support
- I/O virtualization technologies like Intel VT-d and AMD IOMMU allow safe and efficient device sharing between VMs.

5.3 Virtualization of resources

Virtualization can be applied to various resources, here we show the most relevant for cloud computing.

5.3.1 Memory virtualization

Memory management is a critical challenge in virtualization: guest OS maintains its own page tables assuming it owns physical memory.

> **Definition 5.3** ***(Shadow page tables)***. Shadow Page Tables are maintained by the hypervisor, which keeps synchronized copies of guest page tables, translating guest-virtual to host-physical addresses (but with high overhead).

Definition 5.4 ***(Hardware-assisted page tables).*** Extended Page Tables (EPT, Intel) or Nested Page Tables (NPT, AMD) add a second level of hardware translation, reducing overhead.

These technologies drastically improved performance, enabling scalable virtualization in cloud data centers.

5.3.2 I/O virtualization

Input/Output devices must also be virtualized for guest systems. We have difference approaches:

- **Device emulation**: hypervisor emulates hardware devices, highly portable but slow.
- **Paravirtualized drivers** (e.g. VirtIO): guest OS communicates directly with hypervisor through optimized interfaces, achieving better performance.
- **Direct device assignment with RS-IOV** (single root I/O virtualization): near-native performance by partitioning PCI (Peripheral Component Interconnect) devices, at the cost of flexibility and migration limitations.

5.3.3 Network virtualization

Network virtualization abstracts physical networking resources (switches, routers, links) into Software-Defined logical Networks (SDN). It allows multiple isolated virtual networks to coexist on the same physical infrastructure, enabling multi-tenancy in cloud data centers. **Virtual switches** (Linux bridge, Open vSwitch) **connect VMs or containers within each host** as if they were plugged into a real switch. **Overlay networks** (e.g., VXLAN, GRE tunnels) interconnect these virtual switches **across hosts, creating logical topologies independent of the underlying physical network**.

The benefits of network virtualization include:

- strong isolation
- programmability through SDN controllers
- elastic provisioning
- simplified management of large-scale infrastructures.

5.4 Advantages of virtualization

- **Unified vision of physical resources**
  - resources can be on different physical machines or on the network
- **Centralized management** of hardware and software
- **Workload isolation**. Programs are executed inside the VM and are isolated w.r.t. to the host or other VMs
  - Security
- **Consolidation** of virtualized items
  - Optimization of physical resource usage
- **Easy migration** of virtualized items
- Easy way to implement **Redundancy**.
  - Virtualized items can be easily copied

5.4.1 Workload isolation

Through virtualization, workload isolation is achieved since **all program instructions are fully confined inside a VM**, which leads to improvements in security.

Reliability achieved because software failures inside one VM do not affect others.

Performance control is attained since execution of one VM should not affect the performance of another VM.

5.4.2 Workload migration

Workload migration, a.k.a. **application mobility**, improves:

- hardware maintenance,
- performance,
- fault tolerance.

It is done by allowing the guest OS within a VM to be **suspended, migrated to a different platform**, and **resumed immediately** or **preserved** to be restored at a later date. A VM's state includes a full disk, configuration files, and an image of its memory (RAM).

5.4.3 Consolidation

Virtualization makes it possible to consolidate several virtual machines onto a single physical system, reducing the **total cost of ownership** (TCO), which is an estimate of all the direct and indirect costs involved in acquiring and operating a product or system over its lifetime.

5.5 Taxonomy of hypervisors

We can divide hypervisors into two main categories:

- **Bare-metal hypervisors** (Type 1):
  - run directly on the host's hardware to control the hardware and manage guest operating systems. Examples: VMware ESXi, Microsoft Hyper-V, Xen.
  - The hypervisor communicates directly with system hardware rather than relying on a host operating system.
  - The hypervisor is basically a lightweight OS.
  - **advantages**:
    - Improved I/O performance.
  - **Disadvantages**:
    - More complex to set up and manage.
    - It depends on the hardware platform.
- **Hosted hypervisors** (Type 2):
  - run on a conventional operating system just as other computer programs do. Examples: VMware Workstation, Oracle VirtualBox, Parallels Desktop
  - It allows users to run various guest operating systems side by side to their own applications.
  - **Advantages**:
    - Ease of installation and configuration
    - Unmodified Host OS & Guest OS
    - Run on a wide variety of computers
  - **Disadvantages**:
    - Performance overhead due to the additional layer between the hardware and the virtual machines.

5.6 Virtualization techniques

There are several virtualization techniques available:

- **Software Virtualization**
- **Hardware Assisted Virtualization**
  - The hardware implements specific technologies to support virtualization
  - Full virtualization and Para virtualization can exploit it for performance

Here we focus on software virtualization techniques.

5.6.1 Full Virtualization using binary translation

Provide a virtual machine environment that is a full emulation of the underlying hardware.

- The guest OS does not need to be modified

- Guest OS is not aware the existence of the virtual environment
- Each VM is independent of each other

The hypervisor executes **privileged instructions** by dynamically translating the instruction of guest OS into a sequence of instruction appropriate to execute in real hardware (**Binary Translation**). Unprivileged instructions are executed directly.

5.6.2 Para Virtualization (a.k.a. OS assisted Virtualization)

The guest OS is modified to be aware that it is running in a virtualized environment. Instead of trying to **fully emulate hardware** (as in full virtualization), the guest OS uses **special calls** to interact directly with the hypervisor. These calls are usually called **hypercalls**.

- the guest os are aware of the hypervisor and use special calls to interact with it
- performance boost
- seamless integration (mouse, keyboard, screen)

paravirtualized drivers are optimized, virtualization- aware device drivers that replace slow device emulation with direct, efficient communication between the guest OS and the hypervisor. Example: **VirtIO** (KVM/QEMU): standard for paravirtual I/O devices (network, block storage, etc.).

5.6.3 OS-Level virtualization (Containerization)

The host operating system itself provides multiple isolated execution environments, known as **containers** (or zones, jails, etc.), rather than emulating full hardware machines. Unlike hypervisor-based virtualization, OS-level virtualization shares the **same kernel between all containers**. Each container runs as a set of isolated processes, with its own view of the filesystem, process table, network stack, and resource quotas. **The virtualization capability is provided by the host OS**, which performs all the functions of a fully virtualized hypervisor

- Linux Container (Docker, LXD…)

6 Amazon infrastructure

Amazon AWS is a set of building-block services, designed to work independently, that can work well together:

- They share a common naming convention and authentication
- Minimize internal connections and dependencies

Every activity in AWS can be accessed by making a **web service call**.

6.1 Key concepts

Key concepts:

- EC2 = **Compute**: runs the software.
- S3 = **Object storage**: stores objects accessible through APIs.
- EBS = **Block storage**: provides a disk for the virtual machine.

6.1.1 S3

> **Definition 6.1** *(Object store).* An **object store** is a storage architecture that manages and stores data as objects, rather than as files in a file system or blocks in a traditional storage system. Each object in an object store includes three components:
>
> 1. **Data**: The actual content you are storing, such as a file, video, or image.
> 2. **Metadata**: Descriptive information about the object, like its name, creation date, and other relevant details.
> 3. **Unique Identifier**: A unique ID or key assigned to each object, which allows it to be retrieved without needing to know the location of the data (e.g., file path).

Simple Storage Service (S3) is an **object storage** solution:

- with web interface (REST, SOAP, BitTorrent)
- Unlike file systems with hierarchical directories, object stores use a flat address space where every object is stored at the same level.
- Objects up to 5 Terabyte each
- Data automatically distributed across a minimum of three AZs(Availability)
- Automatic data replication (Durability)
- Can be queried using SQL or processed by Big Data solutions

SLA: 99.99% monthly uptime, no more than 4 minutes per month.

Useful for **Backup** & **Recovery**, **Archiving**, and **Disaster Recovery**.

6.1.2 EBS

> **Definition 6.2** *(Block store).* A **block store** is a storage architecture where data is divided into fixed-size chunks called "blocks." Each block is identified by an address within the storage volume. The operating system or application can read and write individual blocks, while a file system typically organizes them to store and retrieve files. Block storage is commonly used in traditional storage devices, such as hard drives and solid-state drives (SSD), as well as in cloud storage systems.

Elastic Block Store (EBS) Volume:

- An addressable disk volume that can be attached to any running instance (a.k.a. VM) in the same Availability Zone
- HDD and SDD
- The volume can then be formatted, mounted, and used as if it were a local disk drive
- Volumes have a lifetime independent of any particular instance (VM)

- A volume can persist even when no instances are running
- Price from $0.025 GB-Month to $0.124 GB-Month (2008)

6.1.3 Security group

A **security group** defines the allowable set of inbound and outbound network connections for an EC2 instance (VM). Each group is named and consists of a list of protocols, ports, and IP address ranges. A group can be applied to multiple instances, and a single instance can be regulated by multiple groups.

A Security Group acts as a **virtual firewall** for an EC2 instance.

- **Rules**: Specify which network traffic is allowed, based on protocol (e.g., TCP), port (e.g., 443 for HTTPS), and source IP address.
- **Inbound and outbound traffic**: Rules control both incoming and outgoing connections.
- **Flexibility**: A security group can be associated with multiple instances, and an instance can have multiple security groups.

> Security groups are stateful: response traffic for an allowed connection is automatically permitted.

6.2 EC2

EC2 implements the **programmable data center metaphor**, later it will be called **Infrastructure as a code (IaC)**. You can get access to multiple Availability Zones in multiple geographic regions and provision your virtualized data center on demand. Servers, operating system images, firewall entries, load balancers, IP addresses, and disk storage volumes are all available in a matter of seconds, by simply making a web service request "to the cloud".

> **Definition 6.3** *(Amazon Machine Image (AMI)).* Contains the operating system and can also include additional software and layers of your application such as database servers, middleware, web servers,...

- Prebuilt AMIs can be started.
- AMIs can be customized, shared, or even sold.
- Each AMI has a unique ID (e.g., ami-bf5eb9d6)

> **Definition 6.4** *(Instance).* Represents one running copy of an AMI. Any number of copies of the same AMI can be launched.

AWS allows you to allocate static IP addresses and then attach them to your instances. Each instance can have at most one such address attached. When the one instance fails, the elastic IP address is remapped to another VM (if it belong to the same group).

An instance can be running any one of several operating systems, including Linux and Windows Server. When you start up an Amazon EC2 instance you must specify the **instance type**. Modern EC2 instances now list specific details like:

- Number of vCPUs
- Actual processor models (e.g., Intel Xeon, AMD EPYC, AWS Graviton)
- Clock speeds
- Physical cores vs. threads

6.2.1 Ephemeral resources

Ephemeral resources are components that only exist while an instance is running and are lost when the instance stops or terminates. They're not permanent: the data vanishes when the instance goes away.

> EC2 instances are ephemeral!

In case of failures, stored data and state information is generally lost.

For production workloads, implement:

- **Redundancy**: Multiple instances across availability zones
- **Failover**: Automatic instance replacement
- **Recovery**: Regular backups and automation scripts

Best suited for **stateless** application patterns

> The **local disk storage** included with each EC2 instance is ephemeral

The storage will remain intact if a running instance is rebooted, but data is permanently lost when instance terminates. Use persistent storage instead:

- Amazon EBS volumes for block storage
- Amazon S3 for object storage
- Amazon EFS for shared file systems

6.2.2 Persistent Resources

Once allocated to your account, they are expected to remain operational in the face of transient or permanent hardware or software failures. The Amazon implementation of a persistent resource makes uses of **redundancy**, **automated failover**, and **automatic recovery** to provide stable resources. The following EC2 resources are persistent:

- Elastic IP addresses
- EBS volumes
- Security groups
- Elastic Load Balancers
- AMIs stored in S3 or as snapshots of EBS volumes

6.2.3 Pricing model

The pricing model determines how compute capacity is paid for and what flexibility you retain.

$$\text{Cost} = \text{compute usage} + \text{related resources and data transfer}$$

There is no single EC2 price: the estimate depends on the workload, configuration, Region and usage pattern. The **configuration** depends on:

- Instance family and size
- Operating system / software
- Region and tenancy

Additional costs may be incurred for:

- EBS storage and performance
- Data transfer (inbound is free, outbound is charged)
- Public IP addresses
- Monitoring and other services

6.2.3.1 Purchase options

On-Demand:

- Pay for the compute you use; no long-term commitment.
- Best for:
  - learning
  - development
  - variable or uncertain demand.

Reserved Instances

- A 1 or 3 year pricing commitment for a defined EC2 configuration/usage pattern
- availability and flexibility depend on the RI type.

Savings Plans

- Commit to a consistent amount of eligible compute usage for 1 or 3 years in exchange for lower rates.
- Best for: predictable baseline usage with flexibility.

Spot Instances

- Use spare EC2 capacity at a variable discount; AWS can interrupt it when capacity is needed.
- Best for: fault-tolerant batch jobs and workloads that can resume.

Workload pattern	Typical starting point	Why it fits	Main trade-off
New / uncertain	On-Demand	No long-term commitment	Higher unit price than commitment options
Stable baseline	Savings Plans; compare RIs	Discount in exchange for commitment	Commitment may outlast demand
Flexible batch work	Spot	Can reduce compute cost	Capacity can be interrupted
Need capacity guaranteed	Capacity Reservation + suitable pricing	Capacity held in a specified Availability Zone	Reservation is a separate concept from discount

> **Rule of thumb**:
> - flexibility → On-Demand
> - steady commitment → Savings Plans / RIs
> - interruption-tolerant → Spot